Budget App

A Category-Based Budget Tracking System in Python

Abstract

This document presents the design and implementation of a simple budgeting system written in Python. The application models financial categories such as food, clothing, and entertainment, allowing users to track deposits, withdrawals, and transfers between categories. In addition to maintaining transaction ledgers, the system provides a text-based visualization that illustrates the percentage of total spending across categories. The project demonstrates fundamental programming principles including object-oriented design, data organization, and formatted output generation.

Repository Overview

The repository contains a lightweight Python implementation of a budgeting tool designed primarily for educational purposes. The project emphasizes clarity, maintainability, and readable code structure while demonstrating practical usage of classes, lists, and formatted string output.

Key Capabilities

Create financial categories with independent transaction ledgers.

Record deposits with optional transaction descriptions.

Record withdrawals with automatic balance verification.

Transfer funds between categories while maintaining a complete transaction history.

Compute balances dynamically based on recorded transactions.

Generate a vertical percentage bar chart showing category spending distribution.

Repository Structure

```
budget-app/
    budget.py       Core implementation containing the Category class and chart generator
    example.py      Demonstration script showing how to use the application
```

README.md Project documentation for the GitHub repository

System Design

The system is centered around a Category class. Each instance represents a specific budget category and maintains a ledger containing transaction records. Every record is stored as a dictionary containing an amount and a description. Deposits store positive values while withdrawals store negative values.

A separate function, create_spend_chart(), analyzes withdrawal activity across multiple categories and produces a formatted percentage chart. The visualization displays spending distribution in 10-percent increments using a vertical bar representation.

Core Implementation (budget.py)

```
class Category:
    def __init__(self, name):
        self.name = name
        self.ledger = []

    def deposit(self, amount, description=""):
        self.ledger.append({
            "amount": amount,
            "description": description
        })

    def withdraw(self, amount, description=""):
        if self.check_funds(amount):
            self.ledger.append({
                "amount": -amount,
                "description": description
            })
            return True
        return False

    def get_balance(self):
        total = 0
        for item in self.ledger:
            total += item["amount"]
```

```
        return total

    def transfer(self, amount, category):
        if self.check_funds(amount):
            self.withdraw(amount, f"Transfer to {category.name}")
            category.deposit(amount, f"Transfer from {self.name}")
            return True
        return False

    def check_funds(self, amount):
        return amount <= self.get_balance()

    def __str__(self):
        title = self.name.center(30, "*") + "\n"
        items = ""

        for entry in self.ledger:
            description = entry["description"][:23].ljust(23)
            amount = f"{entry['amount']:.2f}".rjust(7)
            items += description + amount + "\n"

        total = f"Total: {self.get_balance()}"
        return title + items + total


def create_spend_chart(categories):
    title = "Percentage spent by category\n"

    withdrawals = []
    total_spent = 0

    for category in categories:
        spent = 0
        for item in category.ledger:
            if item["amount"] < 0:
                spent += -item["amount"]
        withdrawals.append(spent)
        total_spent += spent

    percentages = []
```

```
    for spent in withdrawals:
        percent = int((spent / total_spent) * 100)
        percentages.append(percent - percent % 10)

    chart = title

    for i in range(100, -1, -10):
        line = str(i).rjust(3) + "| "
        for percent in percentages:
            if percent >= i:
                line += "o  "
            else:
                line += "   "
        chart += line + "\n"

    chart += "    " + "-" * (len(categories) * 3 + 1) + "\n"

    names = [c.name for c in categories]
    max_length = max(len(name) for name in names)

    for i in range(max_length):
        line = "     "
        for name in names:
            if i < len(name):
                line += name[i] + "  "
            else:
                line += "   "
        chart += line + "\n"

    return chart[:-1]
```

Example Usage

```
from budget import Category, create_spend_chart

food = Category("Food")
food.deposit(1000, "initial deposit")
food.withdraw(10.15, "groceries")
food.withdraw(15.89, "restaurant")
```

```
clothing = Category("Clothing")
food.transfer(50, clothing)

print(food)
print()
print(create_spend_chart([food, clothing]))
```

Example Output

Running the example script will print the formatted ledger for the Food category followed by a percentage chart showing spending distribution between categories.

Conclusion

This project illustrates how object-oriented design can be used to model financial transactions in a structured way. While intentionally simple, the architecture demonstrates practical concepts such as transaction ledgers, balance verification, data aggregation, and formatted reporting. The project serves as a strong foundation for learning Python class design and structured program output.